

机器人学习

COCO

4/3/2026

1 机器人

机器人操作系统（Robot Operating System, ROS）是一个灵活的软件框架，为机器人软件开发提供了丰富的工具和软件库，可以帮助人们提高开发机器人的效率。

选择 ROS 的一些主要原因（高端功能）：ROS 自带现成的算法。

比如，即时定位与地图构建（Simultaneous Localization and Mapping, SLAM）和自适应蒙特卡罗定位（Adaptive Monte Carlo Localization, AMCL），这两个软件包可用于移动机器人的自主导航。

ROS 的消息传递机制允许在不同节点间进行通信。我们可以用 C++ 或 C 语言对节点进行编程，当然也可以用 Python 或 Java 语言。

ROS 技术社区（<http://answers.ros.org>）

ROS 的主要仿真平台是 Gazebo，Gazebo 仿真器没有内置的编程环境，所有仿真必须在 ROS 下编程完成。与 Gazebo 比较，其他仿真环境（如 V-REP、Webots）有内置的编程环境，可以直接对机器人进行原型设计与编程。而且，这些仿真环境有丰富的图形界面（GUI）工具集，支持各类机器人，并且提供了 ROS 接口。虽然这些相应的工具都是各自平台专用的，但效果也都不错。ROS 中主要有三个级别：文件系统、计算图、社区。

为了能创建、修改、使用 ROS 软件包，我们需要先了解一些常用命令。下面是一些使用 ROS 软件包的常用命令 `catkin_create_pkg` 此命令用于创建一个新软件包。`rospack` 此命令用于获取软件包相关的信息。`catkin_make` 此命令用于在工作区中编译软件包。`rosdep` 此命令将安装一个软件包所需的系统依赖项。

ROS 博客会定期更新与 ROS 社区相关的新闻、照片、视频（<http://www.ros.org/news>）

2 ROS 编程入门

ROS 软件包是 ROS 系统的基本单位。

使用 ROS 软件包的第一个要求是创建 ROS 的 `catkin` 工作区。

话题是两个节点间通信的基本方式。下面我们将创建两个 ROS 节点，一个发布话题，另一个订阅该话题。

节点间的相互通讯通过 `/number` 完成。`demo_topic_publisher` 节点向 `/numbers` 话题发布信息，然后 `demo_topic_subscriber` 节点订阅这个话题

与 ROS 服务一样，在 `actionlib` 中，我们也必须初始化动作规范。这个动作的规范存储在以 `.action` 结尾的文件中。该文件必须保存在 ROS 软件包内的 `action` 文件夹中。`action` 文件包含以下各部分内容。

- Goal（目标）：动作客户端可以发送一个必须由动作服务器来执行的目标。这就类

似于 ROS 服务中的请求。例如，如果机器人手臂关节想从 45 度转动到 90 度，那么这里的目标就是 90 度。

- **Feedback（反馈）**：动作客户端向动作服务器发送目标后，将开始执行回调函数。反馈只是简单地给出回调函数内当前操作的进度。通过使用反馈，我们可以获得当前任务的进度。在前面的例子中，机器人手臂关节必须移动到 90 度，在这种情况下，反馈就是机械臂从 45 度转动到 90 度之间的中间值。
- **Result（结果）**：完成目标后，动作服务器将发送完成的最终结果，它可以是计算结果或者一个确认。在前面的例子中，如果关节转动到 90 度，则完成了目标任务，结果就是可以表示目标完成的任何形式。

话题（topic）、服务（service）和动作库（actionlib）可应用于不同的场景。我们知道话题是单向通信的，服务是具备请求/应答功能的双向通信，动作库是 ROS 服务的一种变化形式，我们可以在需要时取消在服务器上运行的进程。

3 在 ROS 中为 3D 机器人建模

机器人制造的第一个阶段是设计和建模。这里我们可以使用 CAD 软件对一个机器人进行设计和建模，例如 AutoCAD、SOLIDWORKS 和 Blender。机器人建模的主要目的之一就是仿真。

将讨论两类机器人的设计。一个是七自由度（7 Degrees of Freedom, 7- DOF）机械手臂，另一个是差速驱动机器人。

可以使用 URDF 来定义机器人模型、传感器和工作环境，使用 URDF 解析器对其进行解析。

设计好 URDF 后，可以在 RViz 上查看它。我们可以创建一个 view_demo.launch 启动文件，并将下面的代码存放到该文件中，然后将该文件放入 launch 文件夹。